

Streszczenie wykonawcze

Projekt **NexusAI** to aplikacja monolityczna składająca się z frontendowego klienta React/Vite oraz backendu Django/REST. Backend zawiera aplikację „ChatApp” oraz moduły serwerowe do obsługi LLM (m.in. Hugging Face, LangChain). Obecna architektura wykorzystuje SQLite jako bazę danych oraz lokalny Redis do cache’owania ¹ ². Takie rozwiązania nie skaluje się dobrze w środowisku produkcyjnym – SQLite blokuje całą bazę przy zapisie ³, co prowadzi do wąskich gardeł przy dużym obciążeniu. Ponadto logika biznesowa (np. metoda `Huggingpt` w widokach) jest silnie sprzężona z niskopoziomowymi wywołaniami LLM i obsługą wątków ⁴ ⁵.

Aby aplikacja była wysoce skalowalna, zalecamy: rozdzielenie warstw (np. oddzielne serwisy do zadań LLM), przejście na produkcyjną bazę danych (PostgreSQL), wykorzystanie mikroserwisów lub kolejek zadań (np. Celery), oraz pełną konteneryzację z automatycznym skalowaniem (Kubernetes). Należy także zabezpieczyć aplikację, wdrożyć CI/CD, strategie testowania (unit, integracyjne, obciążeniowe), a także monitoring i zarządzanie sekretami. Poniższy raport szczegółowo opisuje obecną architekturę, identyfikuje wąskie gardła i przedstawia konkretne zmiany do wprowadzenia.

1. Architektura systemu i mapa modułów

Obecna struktura kodu wygląda następująco:

- **frontend** – projekt React (NexusFrontEnd) uruchamiany z Vite (pliki `package.json`, `vite.config.js` w katalogu `frontEnd/NexusFrontEnd`) ⁶ ⁷. Komponenty komunikują się z backendem poprzez REST (np. `/chat/`, `/threads/`).
- **backend** – monolityczna aplikacja Django w katalogu `backend/NexusAI`. Zawiera:
 - projekt Django **ChatBot** (plik `ChatBot/settings.py`, zawiera konfigurację aplikacji, bazy itp.) ²,
 - aplikację Django **ChatApp** (modele, widoki REST, serializery).
 - Moduł `Server` z podmodułami `Agents` (definicje wejść LLM) oraz `llm` (klasy obsługi modeli HF).

Cały backend uruchamiany jest przez `manage.py` i skrypt `main.py` (wrapper do `runserver`). Jednostronny serwer HTTP odpowiada na żądania z UI.

```
flowchart LR
    subgraph Frontend
        UI[Klient React/Vite]
    end
    subgraph Backend
        API[Django REST API]
        DB[(SQLite)]
        Cache[(Redis)]
        LLM_mod[Serwisy LLM (HuggingFace, Stable Diffusion)]
    end
    UI -->|HTTP| API
    API --> DB
```

```
API --> Cache
API --> LLM_mod
LLM_mod --> API
note right of DB: Obecnie SQLite 2
note right of Cache: Redis w pamięci 1
```

Kluczowe moduły backendu:

- `ChatApp.models` – definicje modeli (m.in. `User`, `Thread`, `Chat`, `AIResponse`, `AIImage`, `Audio` itp.) przechowujących historię czatu i multimedia.
- `ChatApp.views` – kontrolery REST (klasy dziedziczące po `APIView`), np. `Huggingpt` obsługuje komunikację z LLM, zapisuje odpowiedzi AI do bazy, generuje obrazy przy pomocy `InferenceClient` ⁸ ⁹. Widoki te silnie integrują się z kodem serwera LLM (funkcjami `chat_huggingface`, `chat_llama`).
- `Server/awesome_chat.py` – główna logika wielowątkowego przetwarzania zadań LLM (funkcje `chat_llama`, `chat_huggingface`, `run_task` itd.) ¹⁰ ¹¹.
- `Server/llm/huggingface.py` – definicja klasy `HuggingFaceChatModel` wykorzystującej LangChain oraz HF Hub (ponad 200 linii).
- `Server/Agents/InputModel.py` – definicje modeli wejściowych dla narzędzi LLM (Pydantic) ¹² ¹³.
- Konfiguracja Django (`INSTALLED_APPS`, middleware, ustawienia JWT, CORS, cache Redis) znajduje się w `ChatBot/settings.py` ¹⁴ ¹.

Frontend jest prostą aplikacją React korzystającą m.in. z Axios, JWT, Bootstrap ⁶ ⁷. Komunikacja opiera się na endpointach Django (zwracających JSON). Nie ma natomiast oddzielnego serwera media – obrazy generowane są lokalnie w folderze `chat_images/` i serwowane przez Django.

2. Graf zależności, sprzężenia i wąskie gardła

Graf zależności

Moduły są silnie sprzężone w ramach monolitu: `ChatApp` bezpośrednio wywołuje funkcje z `Server` (np. `chat_huggingface`, `InferenceClient`) ⁸ ⁵. Zarówno API, jak i modele danych współdzielą tę samą bazę SQLite, a statyczne pliki obrazu przechowywane są w lokalnym systemie plików. Brak jest warstwy pośredniczącej (np. broker kolejki) między frontendem, backendem i serwisami LLM.

```
graph LR
    subgraph Django
        DB[(SQLite)]:::db
        UserMD[User (model)]:::mod
        ThreadMD[Thread (model)]:::mod
        ChatVD[Huggingpt (view)]:::view
        ChatVD -->|zapis w DB| DB
        ChatVD -->|u\u017cywa LLM| LLMmod
        UserMD --> DB
        ThreadMD --> DB
    end
    subgraph LLM_Services
```

```

HFChat[HuggingFaceChatModel]::mod
StableDiff[StableDiffusion API]::mod
HFChat -->|API call| HFService[(HuggingFace API)]
StableDiff -->|API call| HFService
end
classDef db fill:#fdf5e6
classDef mod fill:#f0f8ff
classDef view fill:#e6ffe6

```

Sprzężenia i pojedyncze punkty awarii

- **Baza danych:** Użycie plikowego SQLite sprawia, że każdy zapis blokuje całą bazę ³. To wielki problem przy wielu jednoczesnych zapytaniach (wielu użytkowników). SQLite nie wspiera efektywnie konkurencyjnych zapisów ¹⁵. Baza jest pojedynczym wąskim gardłem – nie pozwala na horyzontalny wzrost przepustowości ani replikację.
- **Logika LLM w monolicie:** Metoda `Huggingpt` w widoku `ChatApp.views` uruchamia sesję LLM i generuje obraz synchronicznie ⁹. Cały proces (flagi, pętle w `awesome_chat.py`) jest wykonywany w żądaniu HTTP. Długi czas odpowiedzi blokuje worker w WSGI/Gunicorn, ograniczając skalowalność i wydajność. Brak asynchroniczności (np. Celery) oznacza, że ciężkie wywołania LLM mogą przeciążyć proces serwera.
- **Cache Redis:** Choć skonfigurowany, obecny kod go nie wykorzystuje (poza domyślną sesją Django). Musimy ocenić, czy i jak używać Redis (np. do sesji, TTL, ograniczeń itp.).
- **Statyczne zasoby:** Obrazy AI są zapisywane lokalnie (`AIImage.image.save`), co powoduje zależność od dysku lokalnego i problem przy podziale na wiele replik. Brak CDN lub zewnętrznego storage.
- **Singletony w kodzie:** Niektóre zasoby (np. token do API LLM, pipeline Stable Diffusion) są twardo zakodowane lub pobierane w locie, co utrudnia profilowanie i wymaga dostępności łącza do HF. Błąd w zewnętrznym API (przerwa) może zablokować funkcję czatu.
- **Brak warstwy frontend-backend:** Bez pośrednika (np. niezależnego serwisu websocket lub mikroserwisu LLM) cały ruch przechodzi przez jeden endpoint, co ogranicza łatwość rozproszonych wdrożeń.

Potencjalne wąskie gardła

- **Baza SQLite** – już opisana, nie nadaje się do środowiska produkcyjnego ³.
- **Długi czas odpowiedzi API** – obliczenia LLM i generacja obrazów trwają, co może blokować inne requesty. W jednej instancji wątkowej (uwzględniając domyślny Gunicorn) zapytania stają w kolejce.
- **Brak równoważenia obciążenia** – nie opisano load balancera czy auto-skalowania w istniejącym kodzie.
- **Brak testów** – obciążenie kodu i brak testów oznacza ryzyko regresji przy modyfikacjach.

3. Refaktoryzacje i wzorce projektowe

Ogólne zasady

- **Modularność i enkapsulacja:** Oddzielić logikę odpowiedzialną za rozmowy AI od logiki API. Stworzyć warstwy usług (services) lub zastosować wzorzec *serwis/kontroler*, gdzie widoki wywołują serwisy biznesowe, a te – funkcje LLM.

- **Wzorzec Repozytorium:** Zamiast bezpośrednio używać `ChatModel.objects.filter(...)`, można wprowadzić warstwę repozytorium (np. `ChatRepository`) realizującą operacje na bazie. Ułatwia to testowanie i zmianę źródła danych.
- **Wzorzec Fabryki dla modeli LLM:** Zamiast tworzyć instancje LLM czy pipeline wewnątrz kodu, wydzielić fabrykę, np. `LLMFactory.get_model(name)`, aby łatwo podmieniać implementacje (HuggingFace, API, lokalnie).
- **Strategia (Strategy):** Dla różnych źródeł modelu (OpenAI, HuggingFace, lokalny service) można użyć wzorca strategii, gdzie konfiguracja wybiera konkretną klasę implementującą wspólny interfejs `ChatModelInterface`.
- **Asynchroniczność / Kolejowanie:** Refaktoryzacja kodu tworzącego obrazy i generującego odpowiedzi AI – przenieść do zadań w tle (np. Celery, RQ). Widok `Huggingpt` powinien jedynie przyjąć zapytanie, zapisać je, wystartować asynchroniczne zadanie i zwrócić `task_id`; użytkownik może później sprawdzać status lub używać WebSocket.
- **DTO / Serializery:** W modelach wejściowych (Pydantic, `InputModel.py`) już użyto Pydantic do walidacji. Podobnie w Django używać serializerów DRF (np. `UserSerializer`) do konwersji JSON ↔ obiekty. Ujednolicić przepływ danych, aby serwis LLM przyjmował uporządkowane dane.
- **Usuwanie duplikacji:** W `settings.py` widać duplikację `'django.middleware.common.CommonMiddleware'`¹⁶ – wyeliminować. Stosować `settings.py` zgodnie z oficjalnymi zaleceniami Django.

Przykładowe poprawki kodu

1. Oddzielenie logiki generowania odpowiedzi AI: Stworzyć np. serwis `AiChatService` zamiast robić wszystko w widoku:

```
- class Huggingpt(APIView):
-     def post(self, request):
-         # logika wywołania LLM i zapisu do DB
-         ...
-         response = chat_llama(...)
-         ai_response.save()
-         return JsonResponse(...)
+ class Huggingpt(APIView):
+     def post(self, request):
+         user_input = request.data.get("message")
+         thread = ... # pobierz wątek z DB
+         # wywołanie serwisu biznesowego
+         result = AiChatService.chat(user=request.user, message=user_input,
thread=thread)
+         return JsonResponse(result)
+
+ # nowa klasa usługi:
+ class AiChatService:
+     @staticmethod
+     def chat(user, message, thread):
+         # walidacja, zapis zapytania...
+         answer, image_url = LlmAdapter.ask(message, thread)
+         # zapisz odpowiedź i obraz
+         AIResponse.objects.create(chat=thread, message=answer)
+         if image_url:
```

```
+         AIImage.objects.create(chat=thread, path=image_url)
+         return {"message": answer, "path": image_url}
```

Taki podział ułatwi testowanie `AiChatService` i wymianę implementacji LLM.

2. Wzorzec Adapter / Fabryka dla LLM: Utworzyć klasę `LlmAdapter` zwracającą odpowiedni model LLM:

```
class LlmAdapter:
    @staticmethod
    def ask(message, thread):
        model_type = os.getenv("LLM_PROVIDER")
        if model_type == "huggingface":
            return HuggingFaceModel.chat(message)
        elif model_type == "openai":
            return OpenAIModel.chat(message)
        else:
            # fallback do lokalnego
            return HuggingFaceModel.chat(message)
```

W efekcie konfiguracja (ENV) decyduje, z którego provider skorzystać, a nie pobieranie tokenów w kodzie widoku ⁸.

3. Przykład refaktoryzacji widoku klasy na ViewSet:

Zamiast wielu klas `APIView`, można wykorzystać `ViewSet` DRF z routerem:

```
# w urls.py
router = DefaultRouter()
router.register(r'threads', ThreadViewSet, basename='thread')
urlpatterns = router.urls + [
    path("chat/", AiChatView.as_view(), name="chat"),
    ...
]
```

Umożliwi to lepsze wykorzystanie standardowych operacji (CRUD) i automatyczną dokumentację (np. Swagger).

4. Użycie wzorca Singleton dla InferenceClient: Zamiast tworzyć nowego klienta HF przy każdej prośbie o obraz ¹⁷, zastosować wzorzec Singleton lub pulę połączeń:

```
class StableDiffusionClient:
    _instance = None
    def __init__(self):
        token = os.getenv("LLAMA_TOKEN")
        self.client = InferenceClient(model="stabilityai/stable-
diffusion-2-1", token=token)
    @classmethod
```

```
def get_instance(cls):
    if not cls._instance:
        cls._instance = StableDiffusionClient()
    return cls._instance
```

Dzięki temu redukujemy koszty inicjalizacji.

4. Infrastruktura i CI/CD

Konteneryzacja i orkiestracja

- **Docker:** Istnieje plik `backend/NexusAI/Dockerfile`. Należy go zoptymalizować (użyć *multi-stage build*). Przykład (w **Dockerfile** w katalogu backend):

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY backend/NexusAI/requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/
lib/python3.11/site-packages
COPY backend/NexusAI /app
ENV DJANGO_SETTINGS_MODULE=ChatBot.settings
EXPOSE 8000
CMD ["gunicorn", "ChatBot.wsgi:application", "--bind", "0.0.0.0:8000"]
```

Dla frontendu należy utworzyć osobny Dockerfile: buduje aplikację React i serwuje ją np. Nginx'em albo jako pliki statyczne Django.

- **Orkiestracja:** Rekomendujemy **Kubernetes** (K8s) – przenośną platformę open source do zarządzania kontenerami ¹⁸. Umożliwi łatwe skalowanie (Horizontal Pod Autoscaler), deklaratywne wdrożenia (Deployment) i zarządzanie stanem (ConfigMaps, Secrets).
- **Autoskalowanie:** W oparciu o obciążenie CPU/RAM HPA automatycznie zwiększy liczbę replik serwisu Django. Przykładowy manifest K8s:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nexusai-backend
spec:
  replicas: 2
  selector:
    matchLabels: {app: nexusai}
  template:
    metadata: {labels: {app: nexusai}}
    spec:
```

```

containers:
  - name: django
    image: myregistry/nexusai-backend:latest
    ports: [{containerPort: 8000}]
    env:
      - name: DJANGO_SETTINGS_MODULE
        value: ChatBot.settings
    resources:
      requests: {cpu: "500m", memory: "512Mi"}
      limits: {cpu: "1000m", memory: "1Gi"}
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: nexusai-backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: nexusai-backend
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 60

```

Podobnie dla frontendu (jeśli serwowany niezależnie) i ewentualnych workerów (np. Celery).

- **Monitoring i logowanie:** Użyć Prometheusa/Grafany do metryk (np. liczba żądań, opóźnienia) oraz ELK/Fluentd do logów aplikacji. Implementacja health check (endpoint `/healthcheck/`) już istnieje i może służyć K8s do readiness/liveness probes.
- **Zarządzanie sekretami:** Przechowywać wrażliwe dane (klucze API, tokeny) poza repozytorium. W K8s użyć **Kubernetes Secrets** lub zewnętrznego rozwiązania jak HashiCorp Vault. Plik `.env` nie powinien być trzymany w repo (powinien być zaszyfrowany lub refaktoryzowany do Secrets).
- **Infrastructure as Code:** Jeśli wdrażamy na chmurze (np. AWS/GCP/Azure), użyć Terraform lub Pulumi do definiowania klastrów Kubernetes, bazy danych RDS/Postgres, Redis itp. np. w Terraform:

```

resource "aws_instance" "k8s_master" {
  # konfiguracja węzła sterującego
}
resource "aws_eks_cluster" "prod" {

```

```
# konfiguracja klastra EKS
}
```

Takie rozwiązanie zapewni odtwarzalność i wersjonowanie infrastruktury.

- **CI/CD:** Rekomendujemy GitHub Actions/GitLab CI (przykład GH Actions poniżej) do automatycznego budowania, testowania i wdrażania. Przykładowy plik `.github/workflows/ci-cd.yaml`:

```
name: CI/CD
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: python-version: '3.11'
      - name: Install dependencies
        working-directory: backend/NexusAI
        run: pip install -r requirements.txt
      - name: Run tests
        working-directory: backend/NexusAI
        run: pytest # zakładamy, że testy zostały przygotowane
      - name: Build & push Docker images
        uses: docker/build-push-action@v4
        with:
          push: true
          tags: myregistry/nexusai-backend:latest
      - name: Deploy to Kubernetes
        uses: azure/k8s-deploy@v3 # lub inny plugin do k8s
        with:
          manifests: |
            deployment.yaml
            hpa.yaml
```

W powyższym przykładzie: przy pushu do `main` najpierw uruchamiamy testy, potem budujemy obrazy i wypychamy je do rejestru kontenerów, a na końcu aktualizujemy wdrożenie w K8s (np. używając GitOps lub bezpośredniego `kubectl apply`).

5. Strategia przechowywania danych, cache i stan

- **Baza danych:** Migracja z SQLite do skalowalnego rozwiązania (PostgreSQL lub innej relacyjnej). Preferujemy PostgreSQL ze względu na wsparcie w Django i łatwość replikacji. Przykładowa konfiguracja w `settings.py`:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': 'nexusbdb',
        'USER': 'nexususer',
        'PASSWORD': get_env_var('DB_PASSWORD'),
        'HOST': 'db.example.com',
        'PORT': '5432',
    }
}
```

W tabeli `Chat`, `AIResponse` itp. dobrze jest dodać indeksy (np. na polach `thread`, `created_at`), co przyspieszy zapytania. Dane multimedialne (obrazy) najlepiej trzymać poza bazą (np. Amazon S3/GCP Storage) i w bazie przechowywać tylko URL lub ścieżkę.

- **Cache:** Używać Redis nie tylko do sesji, ale też jako pamięć podręczną dla kosztownych zapytań. Na przykład: odpowiedzi AI dla identycznych zapytań można cache'ować przez krótki czas (zwłaszcza przy użyciu tego samego kontekstu), aby zredukować powtarzające się wywołania API. Django ma wbudowane *cache framework* z backendem Redis ¹. Można wykorzystać dekorator `@cache_page` dla endpointów mało zmieniających się.
- **Przechowywanie stanu:** Aplikacja powinna być stateless – każda instancja serwera nie zachowuje trwałego stanu między restartami. Użyć trwałej bazy (jak wyżej) i scentralizowanego cache. Kluczowym elementem jest sesja lub uwierzytelnianie: JWT jest już w użyciu, co wspiera brak stanowych sesji po stronie serwera. W przypadku serwera pracującego z Celery zadbać o trwałe broker (np. RabbitMQ lub Redis) i backend wyników (np. Redis).
- **Zarządzanie współbieżnością:** Użyć np. Gunicorn (wiele workerów) oraz Event Loop (uvicorn) jeśli wprowadzimy async. Alternatywnie wdrożyć Celery do równoległych zadań (Generacja obrazów, LLM) w tle. Upewnić się, że ustawienia Django (`DATABASES['default']` `['CONN_MAX_AGE']`) są zoptymalizowane pod wielowątkowość.
- **Projekt API:** Aby wspierać skalę poziomą, API powinno być idempotentne, wspierać stronicowanie list (np. w endpoints `threads/`) i unikać nadmiernych payloadów. Upewnić się, że logika nie trzyma niewolników (locking) po stronie bazy. Dobrą praktyką może być wprowadzenie tokenów CSRF przy operacjach zmiany stanu (choć JWT tu wystarcza).

6. Strategia testowania i rollout/migracja

- **Testy jednostkowe i integracyjne:**
- *Unit tests:* dla każdego modelu i serwisu biznesowego. Np. testować serializery, walidację modeli oraz logikę `AiChatService`.

- **Testy integracyjne:** testować całe widoki API przy użyciu klienta Django lub `pytest-django`, sprawdzając poprawność odpowiedzi JSON, statusów HTTP i modyfikacji bazy.
- **Testy komponentowe frontendu:** np. za pomocą Jest+React Testing Library sprawdzić komponenty form i ścieżkę uwierzytelniania.
- Uruchamiać te testy w każdym buildzie CI (patrz fragment workflow powyżej).
- **Testy obciążeniowe:** Wykorzystać narzędzia takie jak Locust lub JMeter do symulacji ruchu użytkowników (różne scenariusze: głównie GET API, generowanie odpowiedzi, rejestracja nowych użytkowników). Pozwoli to zweryfikować, czy autoskalowanie działa poprawnie.
- **Testy regresyjne:** Każda zmiana w kodzie powinna przechodzić pełny zestaw testów, aby upewnić się, że nie nastąpi regresja. Używać coverage reportów, aby trafiają testy do krytycznych obszarów (np. interfejsy zewnętrzne).
- **Plan migracji:**
 - **Migracja bazy:** Najpierw stworzyć nową instancję Postgresa i wyeksportować dane ze SQLite (np. za pomocą `dumpdata` / `loaddata` lub narzędzia jak `pgloader`). Uaktualnić `settings.py` (ENV) i wykonać migracje (`manage.py migrate`).
 - **Testy po migracji:** Uruchomić testy i porównać funkcjonalność, aby upewnić się, że nowa baza działa jak poprzednia.
 - **Wdrożenie etapowe:** Najpierw wdrożyć na środowisko staging, przetestować obciążenie i powiadomić zespół. Następnie stopniowo przekierowywać ruch produkcyjny (możliwe wdrożenie typu blue-green albo canary).
 - **Rollback:** Mieć przygotowaną kopię zapasową starej bazy. Jeśli wykryjemy poważne problemy, można w prosty sposób wrócić do poprzedniego deploymentu (Kubernetes Deployment umożliwia rollback) i przywrócić ruch do starej bazy.

7. Bezpieczeństwo i zgodność

- **Uwierzytelnianie i autoryzacja:** JWT jest już skonfigurowany. Upewnić się, że klucze prywatne są bezpiecznie przechowywane (np. w Vault). Sprawdzić, czy wymagane pola wejściowe są walidowane (np. w `SignUp` weryfikacja unikalności). Zastosować limitowanie (rate limiting) – w kodzie widać dodaną bibliotekę `django_ratelimit`, ale trzeba ją skonfigurować na endpointy krytyczne (np. logowania) ¹⁴.
- **Bezpieczeństwo transportu:** Użyć TLS/HTTPS (konfiguracja `SECURE_PROXY_SSL_HEADER` jest już w ustawieniach) ¹⁹. Zapewnić bezpieczne nagłówki (HSTS, CSP) w ustawieniach serwera.
- **Bezpieczeństwo danych:** Wszystkie wrażliwe dane (hasła) haszować i solować (Django `make_password()` już użyty) ²⁰. Jeśli przechowujemy dane osobowe (e-mail, profil), stosować politykę prywatności/GDPR: szyfrowanie nośnika lub bazy, usuwanie danych na żądanie użytkownika.
- **Dependency scanning:** Regularnie skanować biblioteki Pythona i JS na podatności (np. Dependabot, Snyk). Na CI zintegrować `npm audit` i `safety` dla Pythona.

- **Bezpieczne przechowywanie kluczy:** Jak wspomniano, wszystkie klucze API (takie jak `LLAMA_TOKEN`) powinny być w zmiennych środowiskowych (Kubernetes Secrets), a nie w plikach.
- **Ochrona przed atakami:** Użyć wbudowanej ochrony Django (CSRF, XSS, SQLi dzięki ORM). Sprawdzić, czy nie ma bezpośrednich użyc `raw SQL`. Monitorować potencjalnie niebezpieczne operacje.
- **Zgodność prawna:** W zależności od charakteru danych (np. jeżeli aplikacja jest dostępna w UE), zapewnić zgodność z RODO. Przykładowo zbierając dane użytkownika (profil), trzeba dać mu prawo do ich usunięcia, a transfer danych do chmur (np. S3) musi być zaufany i szyfrowany.

8. Harmonogram wdrożeń (roadmap)

Poniżej przykładowy plan implementacji z priorytetami, oszacowanym wysiłkiem (w osobodniowych PD) i ryzykiem:

Etap	Działanie	Wysiłek (PD)	Ryzyko	Priorytet
Bazowa infrastruktura	Migracja DB: SQLite → Postgres, konfiguracja nowej DB (Terraform)	3-5	Średnie (dane)	Wysoki
Konteneryzacja	Optymalizacja Dockerfile backend/front, budowa kontenerów	2-4	Niskie	Wysoki
Orkiestracja	Wdrożenie K8s (minikube/EKS/AKS), definicja Deployment, Service	4-6	Średnie (setup)	Wysoki
Logika AI	Refaktoryzacja LLM do asynchronicznych zadań (Celery + Redis/Rabbit)	5-8	Wysokie (złoż.)	Wysoki
Ujednolicenie API	Wzorce design (usługi, fabryka LLM), poprawa widoków DRF	3-5	Średnie	Średni
Frontend	Dodanie testów UI, obsługa błędów (toast), konfiguracja routing	2-3	Niskie	Średni
CI/CD	Konfiguracja pipeline (GitHub Actions/GitLab CI) + testów automatyzacji	2-4	Niskie	Wysoki
Monitoring/Logging	Zintegrowanie Prometheus/Grafana, ELK/Fały do logów	3-5	Średnie	Średni
Bezpieczeństwo	Audyt kodu, skan podatności, wdrożenie HTTPS, aktualizacja bibliotek	2-3	Niskie	Wysoki
Testy obciążeniowe	Stworzenie skryptów obciążeniowych (Locust), analiza wyników	3-4	Niskie	Średni

Poszczególne etapy można równolegle rozwijać w zespole. Kluczowe jest najpierw uodpornić podstawę (baza i kontenery), potem refaktoryzować warstwę AI, a równolegle wdrażać CI/CD i testy. **Wysokim**

priorytetem są zmiany krytyczne dla skalowalności (migracja bazy, kontenery, asynchroniczna obsługa LLM) oraz CI/CD z testami automatycznymi.

9. Tabela porównawcza wybranych rozwiązań

Kategoria	Opcja A	Opcja B	Komentarz
Orkiestracja	Kubernetes	Docker Swarm / ECS	K8s: pełna elastyczność i społeczność ¹⁸ . Swarm prostszy ale słabiej wspierany. ECS (AWS) daje integrację z chmurą.
CI/CD	GitHub Actions	Jenkins	GitHub Actions: niski próg wejścia, łatwa integracja z repo, skalowalna. Jenkins: bardzo konfigurowalny, ale większy overhead (serwer) ²¹ .
Baza danych	PostgreSQL	MongoDB / MySQL	Postgres: stabilny, relacyjny, transakcyjny, wspierający replikację. MySQL: podobnie (GCP). MongoDB (NoSQL) mógłby być opcją dla nieustrukturyzowanych danych (np. embeddingi), ale stawia koszty migracji modelu danych.
Cache	Redis	Memcached	Redis: więcej funkcji (persistencja, pub/sub), rekomendowany w Django ¹ . Memcached: prostszy, ale mniej możliwości.
Uwierzytelnianie	JWT (DRF Simple JWT)	OAuth 2.0 (np. django-allauth)	JWT: lekki, stateless, już w projekcie użyty. OAuth: pełniejszy, ale wymaga serwera autoryzacji; przydaje się przy integracji z zewn. providerami.
Wiadomości/komunikacja	Celery + RabbitMQ/Redis	Apache Kafka	Celery: znany w Django, łatwy do wdrożenia. Kafka: bardziej na strumieniowanie danych, bardziej skomplikowany.

Tabela pokazuje, że preferujemy rozwiązania z niskim wejściem i dużą społecznością (Kubernetes, GitHub Actions, Postgres, Redis). Alternatywy są możliwe (np. AWS ECS zamiast K8s), ale wymagają oceny specyfiki projektu.

10. Diagramy (Mermaid)

Architektura systemu (komponenty)

```

flowchart TB
    User((Użytkownik))
    subgraph WebClient
        A[Przeglądarka React]
    end
  
```

```

subgraph Services
  B[Django REST API]
  C[(PostgreSQL DB)]
  D[(Redis Cache)]
  subgraph LLMservices
    E[Hugging Face API]
    F[Stable Diffusion API]
  end
end
end
User --> A
A -->|HTTP/JSON| B
B --> C
B --> D
B -->|Zapytanie LLM| E
B -->|Generacja obrazu| F
E --> B
F --> B
C -. -> B

```

Graf zależności (moduły)

```

graph LR
  subgraph Backend
    APP[ChatApp (MODELS, VIEWS)]
    SRV[Server (Agents, LLM)]
    APP --> SRV
    APP --> DB[(DB)]
    APP --> Cache[(Redis)]
  end
  subgraph Frontend
    UI[React app]
  end
  UI --> API[API Django]
  API --> APP

```

Przepływ wdrożenia (CI/CD)

```

flowchart LR
  code[Repozytorium Git]
  code --> build[BUDUJ: Docker images]
  code --> test[TESTY: unit, integ, load]
  build --> push[Dociągnięcie do rejestru]
  push --> deploy[K8s Deployment/Update]
  test --> deploy
  deploy --> prod[Środowisko produkcyjne]
  deploy --> staging[Środowisko staging]

```

Diagramy powyższe ilustrują: architekturę związaną z kontenerami i LLM, zależności między modułami, oraz schemat CI/CD.

Źródła

Elementy architektury i kodu przytoczone powyżej pochodzą bezpośrednio z repozytorium NexusAI (moduły `ChatApp` i `ChatBot/settings.py`, `Server/awesome_chat.py`, `InputModel.py`)¹⁴⁸¹². Informacje o Kubernetes zaczerpnięto z oficjalnej dokumentacji Kubernetes (polskiej)¹⁸. Wskazówki dotyczące SQLite oparto na dyskusjach i najlepszych praktykach (np. SQLite scala słabo przy concurrent writes³¹⁵). Inne sugestie architektoniczne bazują na standardach branżowych i dokumentacji Django/REST oraz doświadczeniach autorskich.

¹ ² ¹⁴ ¹⁶ ¹⁹ NexusAI/backend/NexusAI/ChatBot/settings.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/ChatBot/settings.py>

³ ¹⁵ scalability - How Scalable is SQLite? - Stack Overflow

<https://stackoverflow.com/questions/54998/how-scalable-is-sqlite>

⁴ ⁸ ⁹ ¹⁷ ²⁰ NexusAI/backend/NexusAI/ChatApp/views.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/ChatApp/views.py>

⁵ ¹⁰ ¹¹ NexusAI/backend/NexusAI/Server/awesome_chat.py at master · DawoodTouseef/NexusAI · GitHub

https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/Server/awesome_chat.py

⁶ ⁷ NexusAI/frontEnd/NexusFrontEnd/package.json at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/frontEnd/NexusFrontEnd/package.json>

¹² ¹³ NexusAI/backend/NexusAI/Server/Agents/InputModel.py at master · DawoodTouseef/NexusAI · GitHub

<https://github.com/DawoodTouseef/NexusAI/blob/master/backend/NexusAI/Server/Agents/InputModel.py>

¹⁸ Przegląd | Kubernetes

<https://kubernetes.io/pl/docs/concepts/overview/>

²¹ Jenkins vs. GitHub Actions vs. GitLab CI - DEV Community

<https://dev.to/574n13y/jenkins-vs-github-actions-vs-gitlab-ci-2k35>